In [9]:
```python
#Janvi Yadav B-04
#EXP6

import numpy as np
import random
```

In [2]:
```python
# 0 = Free Path
# 1 = Wall
# 2 = Goal

maze = np.array([
    [0, 0, 0, 1, 2],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
])

print("Maze Environment:")
print(maze)
```

```
Maze Environment:
[[0 0 0 1 2]
 [1 1 0 1 0]
 [0 0 0 0 0]
 [0 1 1 1 0]
 [0 0 0 0 0]]
```

In [3]:
```python
# Maze size
rows, cols = maze.shape

# Possible Actions
actions = ["Up", "Down", "Left", "Right"]

# Q-Table Initialization
q_table = np.zeros((rows, cols, len(actions)))

# Learning Parameters
alpha = 0.8      # Learning Rate
gamma = 0.9      # Discount Factor
epsilon = 0.2    # Exploration Rate
episodes = 500

print("Maze Size:", rows, "x", cols)
print("Actions:", actions)
print("Q-table Shape:", q_table.shape)
```

```
Maze Size: 5 x 5
Actions: ['Up', 'Down', 'Left', 'Right']
Q-table Shape: (5, 5, 4)
```

In [4]:
```python
# Reward Function
def get_reward(state):
    r, c = state

    if maze[r][c] == 2:
        return 100      # Goal Reward
    else:
        return -1       # Penalty for every move
```

```
# Movement Function
def move(state, action):
    r, c = state

    if action == 0 and r > 0:
        r -= 1          # Up
    elif action == 1 and r < rows - 1:
        r += 1          # Down
    elif action == 2 and c > 0:
        c -= 1          # Left
    elif action == 3 and c < cols - 1:
        c += 1          # Right

    # Stay in same position if wall
    if maze[r][c] == 1:
        return state

    return (r, c)

print("Reward and Move Functions Created Successfully")
```

```
Reward and Move Functions Created Successfully
```

In [5]:
```
for episode in range(episodes):

    state = (0, 0)     # Start Position

    while maze[state] != 2:

        # Exploration or Exploitation
        if random.uniform(0,1) < epsilon:
            action = random.randint(0,3)
        else:
            action = np.argmax(q_table[state])

        next_state = move(state, action)

        reward = get_reward(next_state)

        old_value = q_table[state][action]

        next_max = np.max(q_table[next_state])

        # Q-Learning Formula
        q_table[state][action] = old_value + alpha * (
            reward + gamma * next_max - old_value
        )

        state = next_state

print("Training Completed Successfully")
```

```
Training Completed Successfully
```

In [6]:
```
print("Q-Table:\n")

print(np.round(q_table,2))
```

```
Q-Table:

[[[ 37.35  37.35  37.35  42.61]
  [ 42.61  42.61  37.35  48.46]
  [ 48.46  54.95  42.61  48.46]
  [  0.     0.     0.     0.  ]
  [  0.     0.     0.     0.  ]]

 [[  0.     0.     0.     0.  ]
  [  0.     0.     0.     0.  ]
  [ 48.46  62.17  54.95  54.95]
  [  0.     0.     0.     0.  ]
  [100.    79.1   89.    89.  ]]

 [[ -2.84  -2.98  29.9   54.86]
  [ 54.5   43.49  47.05  62.17]
  [ 54.95  62.17  54.95  70.19]
  [ 70.19  70.19  62.17  79.1 ]
  [ 89.    70.19  70.19  79.1 ]]

 [[ -3.01  -2.91  -3.02  -2.99]
  [  0.     0.     0.     0.  ]
  [  0.     0.     0.     0.  ]
  [  0.     0.     0.     0.  ]
  [ 79.1   58.43  -0.8   69.63]]

 [[ -3.24  -2.96  -2.84  -2.83]
  [ -2.35  -2.81  -2.93  -2.33]
  [ -2.21  -1.68  -2.59  -1.8 ]
  [ -1.54  -1.54  -1.65  44.52]
  [ 70.02  36.83  25.52  -1.54]]]
```

In [7]:

```python
state = (0,0)

path = [state]

while maze[state] != 2:

    action = np.argmax(q_table[state])

    state = move(state, action)

    path.append(state)

print("Optimal Path:")
print(path)
```

```
Optimal Path:
[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (1, 4), (0, 4)]
```

In [8]:

```python
maze_display = maze.astype(str)

for r,c in path:
    if maze_display[r][c] == "0":
        maze_display[r][c] = "*"

maze_display[0][0] = "S"   # Start
maze_display[0][4] = "G"   # Goal

print("Maze with Learned Path:\n")
```

```
for row in maze_display:
    print(" ".join(row))
```

```
Maze with Learned Path:

S * * 1 G
1 1 * 1 *
0 0 * * *
0 1 1 1 0
0 0 0 0 0
```